

REINFORCEMENT LEARNING TUTORIAL

从理论到 PyTorch 实战 · 全栈深度算法精讲

强化学习 全面教程

覆盖 MDP、Q-Learning、DQN、Policy
Gradient、

Actor-Critic、PPO、SAC、Rainbow 到 RLHF

本教程系统讲解强化学习的数学基础、核心算法与工程实践，配套 12 份基于 PyTorch + Gymnasium 的可运行代码。从贝尔曼方程到 PPO、SAC，再到 RLHF 与离线 RL，构建从入门到前沿的完整知识体系。

PyTorch 2.x

Gymnasium

14 章

含代码

VERSION v1.0 · 2026.08

AUDIENCE ML 工程师 · 研究生 · 算法爱好者

CODE rl_tutorial_code/ 目录下 12 份脚本

目录

第一章 强化学习入门与核心概念	5
1.1 什么是强化学习	5
1.2 核心概念与术语	5
1.3 与监督学习、无监督学习的对比	6
1.4 RL 的分类体系	6
1.5 经典应用场景	6
1.6 第一个 RL 程序：环境交互循环	7
第二章 数学基础：MDP 与贝尔曼方程	8
2.1 MDP 的形式化定义	8
2.2 回报与价值函数	8
2.3 贝尔曼方程	9
2.4 FrozenLake：MDP 的实例分析	9
第三章 基于值的方法：动态规划	10
3.1 策略评估	10
3.2 策略改进	10
3.3 策略迭代	10
3.4 值迭代	11
3.5 DP 的局限与现代意义	11
第四章 蒙特卡洛方法	12
4.1 MC 预测	12
4.2 MC 控制	12
4.3 Blackjack 实战	12
4.4 MC 的优缺点	13
第五章 时序差分学习：SARSA 与 Q-Learning	13
5.1 TD(0) 预测	13
5.2 SARSA：on-policy TD 控制	13

5.3 Q-Learning: off-policy TD 控制	14
5.4 经典对比: CliffWalking	14
5.5 期望 SARSA 与算法选择	14
5.6 TD(λ): 在 MC 与 TD 之间插值	15
第六章 函数逼近与 DQN	15
6.1 函数逼近基础	15
6.2 DQN 的三大创新	15
6.3 DQN 完整算法	16
6.4 训练技巧与超参数	17
6.5 DQN 的局限	17
第七章 策略梯度方法: REINFORCE	17
7.1 策略目标函数	18
7.2 策略梯度定理	18
7.3 REINFORCE 算法	18
7.4 减小方差: 基线与优势	19
7.5 REINFORCE 的局限与改进方向	19
第八章 Actor-Critic 与 A2C/A3C	19
8.1 AC 框架的核心思想	20
8.2 A2C: 同步 Actor-Critic	20
8.3 A3C: 异步并行	20
8.4 GAE: 广义优势估计	21
8.5 A2C 实现要点	21
第九章 PPO: 现代 RL 工业标准	21
9.1 从 TRPO 到 PPO	21
9.2 PPO-Clip 损失函数	21
9.3 PPO 完整目标	22
9.4 PPO 训练流程	22
9.5 超参数选择	23
9.6 PPO 的工程实践	23

第十章 SAC 与连续控制	24
10.1 最大熵 RL	24
10.2 SAC 三大创新	24
10.3 SAC 完整算法	25
10.4 SAC 在 Pendulum 上的表现	25
10.5 SAC vs PPO vs DDPG/TD3	25
第十一章 Rainbow DQN 与改进技巧	26
11.1 Double DQN: 解决 Q 值过估	26
11.2 Dueling Network: 分离 V 与 A	26
11.3 Prioritized Experience Replay	27
11.4 其他三项改进	27
11.5 实战与对比	27
第十二章 前沿主题: RLHF、世界模型、离线 RL、MARL	28
12.1 RLHF: 驱动大模型对齐	28
12.2 世界模型: Dreamer 系列	28
12.3 离线 RL: 从固定数据集学习	29
12.4 多智能体 RL: MARL	29
12.5 大模型时代的 RL 新方向	29
第十三章 算法对比与工程实践	30
13.1 算法选择决策树	30
13.2 算法综合对比	30
13.3 超参数经验	31
13.4 训练曲线分析	31
13.5 工程实践要点	32
13.6 算法对比的可视化思维	32
第十四章 调试、调参与常见问题排查	32
14.1 训练不收敛: 可能的原因	32
14.2 Q 值爆炸	33

14.3 策略熵过早坍缩	33
14.4 Gymnasium 版本兼容	33
14.5 GPU 显存爆炸	34
14.6 探索-利用失衡	34
14.7 评估与对比的常见误区	34
14.8 调试 checklist	35

第一章 强化学习入门与核心概念

强化学习（Reinforcement Learning, RL）是机器学习的三大范式之一，与监督学习和无监督学习并列。其核心思想是：智能体（Agent）通过与环境（Environment）反复交互，根据环境反馈的奖励信号（Reward）不断调整自身行为策略，最终学会在复杂不确定环境下做出能最大化长期累积奖励的决策。这种“试错 + 反馈”的范式天然适合于序列决策问题——从棋类博弈、机器人控制到推荐系统、自动驾驶，再到近年来驱动 ChatGPT 等大语言模型对齐人类偏好的 RLHF 技术，强化学习无处不在。

本章将系统介绍强化学习的基本术语、与其他机器学习范式的区别、问题分类以及典型应用场景，帮助读者建立整体认知框架。后续章节会在此基础上逐步深入到具体算法和工程实现。

1.1 什么是强化学习

强化学习的本质是一个“决策优化”问题：智能体在时刻 t 观察到环境状态 s_t ，依据当前策略 π 选择动作 a_t ，环境接收动作后转移到新状态 s_{t+1} 并返回奖励 r_{t+1} 。智能体的目标是学习一个策略 π^* ，使得在长期交互中获得的累积奖励期望最大。这种“延迟反馈”特性是 RL 与监督学习最大的差异——监督学习中每个样本都有即时的“正确答案”，而 RL 中智能体可能要等多步之后才知道当初的某个动作是否正确。

强化学习的另一个核心特征是“探索与利用”（Exploration vs. Exploitation）的权衡。智能体既需要利用已知的高奖励行为（exploitation），又需要尝试未知行为以发现潜在更优策略（exploration）。这个困境在监督学习中并不存在，但在 RL 中几乎是所有算法都必须显式或隐式处理的问题。常见的探索策略包括 ϵ -greedy、Boltzmann 探索、UCB 以及基于噪声网络的方法等。

1.2 核心概念与术语

在深入算法之前，必须先掌握强化学习的基本词汇。以下术语贯穿全书后续章节。

- **智能体（Agent）**：决策主体，负责观察状态、选择动作、更新策略。
- **环境（Environment）**：智能体之外的万物，定义状态转移规则与奖励函数。
- **状态（State, s ）**：环境在某时刻的描述，可以是完全可观察的（如棋盘），也可以是部分可观察的（如打牌时不知道对手手牌）。
- **动作（Action, a ）**：智能体可执行的操作，可分为离散（左/右）或连续（速度大小）。
- **奖励（Reward, r ）**：环境对动作的即时反馈标量，是智能体优化的唯一信号。
- **策略（Policy, π ）**：从状态到动作的映射，可以是确定性的 $a = \pi(s)$ ，也可以是随机的 $\pi(a|s)$ 。
- **轨迹（Trajectory/Trajectory）**：一个回合内状态-动作-奖励序列 $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ 。
- **回报（Return, G_t ）**：从时刻 t 起的累积折扣奖励 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ 。
- **价值函数（Value Function）**：状态或状态-动作对的长期期望回报，是 RL 评估的核心量。

- **模型 (Model)**：环境的动力学描述 $P(s'|s,a)$ 与奖励函数 $R(s,a)$ 。

1.3 与监督学习、无监督学习的对比

理解 RL 的最好方式是与熟悉的监督学习对比。监督学习有标注好的 (x, y) 对，模型直接学习从输入到输出的映射；而 RL 中没有"正确动作"标签，智能体只能通过尝试不同动作并观察奖励来间接学习。监督学习的损失通常是预测与标签的差异，而 RL 的目标函数是期望累积奖励。此外，监督学习的样本通常是独立同分布的 (i.i.d.)，但 RL 中智能体产生的数据具有强时间相关性，这正是经验回放 (Experience Replay) 等技术存在的原因。

与无监督学习相比，RL 有明确的优化目标（奖励），而非寻找数据的隐藏结构。三者之间的关系可以这样概括：监督学习是"教模型做什么"，无监督学习是"让模型发现规律"，而强化学习是"让模型自己学会做什么"。这种自驱动特性使得 RL 在很多复杂决策场景中独树一帜。

1.4 RL 的分类体系

强化学习算法可从多个维度分类，掌握这些分类有助于在选择算法时快速定位。

1.4.1 按是否学习环境模型分类

- **基于模型 (Model-based)**：先学习环境模型 $P(s'|s,a)$ ，再用模型规划。代表：Dyna、MuZero、Dreamer。
- **无模型 (Model-free)**：不显式学习模型，直接从经验中学习。代表：DQN、PPO、SAC。

基于模型方法样本效率高（可"想象"经验），但模型误差会被放大；无模型方法更通用但样本需求大。现代 RL 在低数据场景（如机器人）倾向基于模型，在高维复杂场景（如 Atari）倾向无模型。

1.4.2 按学习的对象分类

- **基于值 (Value-based)**：学习 $Q(s,a)$ ，间接得到策略。代表：Q-Learning、DQN。适合离散动作。
- **基于策略 (Policy-based)**：直接参数化策略 $\pi_{\theta}(a|s)$ 。代表：REINFORCE、TRPO。适合连续动作。
- **Actor-Critic**：同时学策略与价值。代表：A2C、PPO、SAC。集大成者。

1.4.3 按策略更新方式分类

- **On-policy**：用当前策略产生的数据更新当前策略。代表：SARSA、REINFORCE、PPO。
- **Off-policy**：可用其他策略产生的数据更新当前策略。代表：Q-Learning、DQN、SAC。

Off-policy 通常样本效率更高（数据可重复使用），但训练可能不稳定；On-policy 更稳定但样本效率低。实践中：连续控制首选 PPO 或 SAC；离散控制首选 DQN 系列。

1.5 经典应用场景

强化学习的应用范围极其广泛，下面列举几个有代表性的领域，每个领域都有其独特的挑战与解决方案。

- **游戏 AI**：AlphaGo（围棋）、AlphaStar（星际争霸 2）、OpenAI Five（Dota 2），这些突破展示了 RL 在复杂策略博弈中的潜力。
- **机器人控制**：四足机器人行走、机械臂抓取、灵巧手操作。需要解决高维连续控制、稀疏奖励、Sim-to-Real 等难题。
- **自动驾驶**：车道保持、变道决策、路口通行。安全约束是关键挑战。
- **推荐系统**：新闻推荐、广告投放、商品推荐。RL 能优化长期用户参与度而非短期点击率。
- **大模型对齐（RLHF）**：用 RL 微调 LLM，使其行为符合人类偏好。这是 ChatGPT 成功的关键技术。
- **算法交易**：动态投资组合管理、做市策略、执行优化。需处理非平稳市场与极稀疏信号。

1.6 第一个 RL 程序：环境交互循环

理论讲完，让我们用一段最简单的代码建立直观感受。下面是使用 Gymnasium 库与 CartPole-v1 环境的最小交互循环。CartPole 是 RL 教学的"Hello World"：一根杆子立在小车上，目标是通过左右移动小车让杆子保持直立，每坚持一步得 1 分，最多 500 分（v1 版本）。

```
import gymnasium as gym
import numpy as np

env = gym.make("CartPole-v1")
obs, _ = env.reset(seed=42)
total_reward = 0.0
terminated, truncated = False, False

while not (terminated or truncated):
    action = np.random.randint(0, 2) # 随机策略
    obs, reward, terminated, truncated, _ = env.step(action)
    total_reward += reward

print(f"随机策略得分: {total_reward}")
# 随机策略通常只得 10-30 分；学完本教程后用 DQN/PP0 可达 500 分
```

这段代码揭示了 RL 的核心循环：观察（observe）→ 决策（act）→ 执行（step）→ 反馈（reward）。注意 Gymnasium 0.26+ 的 API 改动：reset() 返回 (obs, info) 元组，step() 返回 5 元组 (obs, reward, terminated, truncated, info)，其中 terminated 表示自然终止（如杆倒了），truncated 表示因时间限制等外部条件终止。这是新手最常踩的坑。

[技巧] Gymnasium 是 OpenAI Gym 的维护分支，2022 年起由 Farama Foundation 接管。旧代码中的 import gym 应迁移到 import gymnasium as gym。环境 API 也有重大变化，详见第八章工程实践部分。

本教程配套代码包中，ch01_env_loop.py 是本章的完整可运行示例。建议读者先运行此脚本，观察随机策略在 CartPole 上的表现，再开始学习后续章节的算法。从下一章开始，我们将进入数学基础，正式建立强化学习的理论框架。

第二章 数学基础：MDP 与贝尔曼方程

马尔可夫决策过程 (Markov Decision Process, MDP) 是强化学习的数学语言。几乎所有现代 RL 算法都可以视为在 MDP 框架下的不同求解策略。本章将系统介绍 MDP 的定义、价值函数、贝尔曼方程以及最优性原理。这些数学基础是理解后续所有算法（从 Q-Learning 到 PPO）的必备前提。

2.1 MDP 的形式化定义

一个马尔可夫决策过程由五元组 (S, A, P, R, γ) 定义：

- S ：状态空间 (State Space)，所有可能状态的集合。
- A ：动作空间 (Action Space)，所有可执行动作的集合。
- $P(s'|s, a)$ ：状态转移函数，表示在状态 s 执行动作 a 后转移到 s' 的概率。
- $R(s, a)$ ：奖励函数，在状态 s 执行动作 a 后获得的期望奖励。
- $\gamma \in [0, 1)$ ：折扣因子，权衡当前与未来奖励的重要性。

MDP 的核心假设是**马尔可夫性**：下一状态 s_{t+1} 和奖励 r_{t+1} 只依赖于当前状态 s_t 和动作 a_t ，而与更早的历史无关。用数学语言表述为：

$$P(s_{t+1} = s' \mid s_0, a_0, \dots, s_t, a_t) = P(s' \mid s_t, a_t)$$

这个假设极大地简化了问题：智能体不需要记住完整历史，只需要当前状态就能做出最优决策。当然，现实世界中很多问题并不严格满足马尔可夫性（如打牌时不知道对手手牌），此时可以通过引入“部分可观察 MDP” (POMDP) 来处理，但底层算法仍基于 MDP。

2.2 回报与价值函数

强化学习的优化目标是最大化期望回报。回报 G_t 定义为从时刻 t 起的折扣累积奖励：

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

折扣因子 γ 的作用至关重要。 γ 越接近 1，智能体越重视长远回报（“远视”）； γ 越接近 0，智能体越贪婪于即时奖励（“近视”）。实践中 γ 通常取 0.9–0.99。 γ 的另一作用是保证无限时序问题中回报有界：当奖励有界且 $\gamma < 1$ 时， G_t 收敛。

基于回报，定义两个核心价值函数：

2.2.1 状态价值函数 $V^\pi(s)$

$V^\pi(s)$ 表示在策略 π 下，从状态 s 出发的期望回报：

$$V^\pi(s) = E_\pi[G_t \mid s_t = s]$$

2.2.2 动作价值函数 $Q^\pi(s, a)$

$Q^\pi(s, a)$ 表示在策略 π 下，从状态 s 执行动作 a 后的期望回报：

$$Q^\pi(s, a) = E_\pi[G_t \mid s_t = s, a_t = a]$$

两者的关系是： $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$ 。直观上， V 是"这个状态有多好"， Q 是"在这个状态下执行这个动作有多好"。基于值的算法（如 DQN）学 Q ，基于策略的算法直接学 π ，Actor-Critic 同时学两者。

2.3 贝尔曼方程

贝尔曼方程（Bellman Equation）是 RL 的核心递推关系，它将价值函数分解为"即时奖励 + 下一状态的价值"，是几乎所有 RL 算法的理论基础。

2.3.1 贝尔曼期望方程

对于 V^π ，有：

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^\pi(s')]$$

对于 Q^π ，有：

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')]$$

这两个方程的直观含义：当前状态的价值 = 即时奖励的期望 + 下一状态价值的折扣期望。这是一个递归关系，理论上可以通过迭代求解（这就是策略评估的基础）。

2.3.2 贝尔曼最优方程

当策略是最优策略 π^* 时，价值函数满足贝尔曼最优方程：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

最优策略可以从 Q^* 直接推出： $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$ 。这正是 Q-Learning 的理论基础——只要学到 Q^* ，就能得到最优策略。贝尔曼最优方程是非线性的（含 $\max$ 操作），不能直接解析求解，但可以通过值迭代等数值方法逼近。

2.4 FrozenLake: MDP 的实例分析

为了让概念具体化，我们以 FrozenLake-v1 为例分析一个完整 MDP。FrozenLake 是 4×4 网格世界，目标是让智能体从起点 S 走到目标 G，途中避开冰洞 H。环境有 `is_slippery` 选项：开启时动作有 1/3 概率横向滑动（模拟冰面湿滑），使问题更具挑战性。

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True)

# 查看 MDP 的完整转移函数
P = env.unwrapped.P
print(P[0][2]) # 状态 0 执行动作 2（向右）
# [(0.333, 0, 0.0, False), (0.333, 1, 0.0, False), (0.333, 0, 0.0, False)]
# 含义：1/3 概率原地不动，1/3 概率向下到状态1，1/3 概率回到0
```

Gymnasium 的 `env.unwrapped.P` 字段直接暴露了环境的 MDP，可以直接用于动态规划求解（见第三章）。但实际中绝大多数环境不会提供 `P`，必须通过交互学习。

配套代码 `ch02_mdp_frozen_lake.py` 实现了完整的策略评估代码，可以直接运行观察均匀随机策略下的状态价值函数。建议读者运行后对比不同 γ 值下的 V^π ，直观感受折扣因子的影响。

第三章 基于值的方法：动态规划

动态规划 (Dynamic Programming, DP) 是 RL 的"祖先"——它要求完全已知 MDP 模型，在大多数实际问题中并不实用。但 DP 提供了 RL 的理论基础与思想原型：所有现代 RL 算法都可以视为"在不知道 MDP 时如何近似 DP"的不同方案。本章介绍策略评估、策略改进、策略迭代与值迭代四个核心算法。

3.1 策略评估

策略评估 (Policy Evaluation) 解决的问题是：给定策略 π ，求其价值函数 V^π 。根据贝尔曼期望方程，可以构造迭代：

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

这被称为"迭代策略评估"。理论上当 $k \rightarrow \infty$ 时 V_k 收敛到 V^π 。实践中用一个小的阈值 θ (如 $1e-8$) 检测收敛：当 $\max |V_{k+1}(s) - V_k(s)| < \theta$ 时停止。该算法的核心代码非常简洁——遍历所有状态、所有动作、所有可能的下一状态，计算期望。

```
def policy_eval(env, policy, gamma=0.99, theta=1e-8):
    nS = env.observation_space.n
    V = np.zeros(nS)
    while True:
        delta = 0
        for s in range(nS):
            v = 0
            for a, pi_a in enumerate(policy[s]):
                for (p, s_next, r, done) in env.unwrapped.P[s][a]:
                    v += pi_a * p * (r + gamma * V[s_next] * (1 - done))
            delta = max(delta, abs(v - V[s]))
            V[s] = v
        if delta < theta:
            break
    return V
```

3.2 策略改进

策略改进 (Policy Improvement) 解决的问题是：给定 V^π ，能否构造一个更好的策略 π' ？答案是肯定的——对每个状态贪心地选择最大化动作价值的动作即可：

$$\pi'(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^\pi(s')]$$

策略改进定理保证：新策略 π' 至少不比原策略 π 差，即 $V^{\pi'}(s) \geq V^\pi(s)$ 对所有 s 成立。如果 $\pi' = \pi$ ，则 π 已是最优策略。这个定理是策略迭代收敛性的理论基础。

3.3 策略迭代

策略迭代 (Policy Iteration) = 策略评估 + 策略改进 的循环:

- 1. 初始化随机策略 π_0
- 2. 评估 π_k 得到 V^{π_k}
- 3. 改进得到 π_{k+1}
- 4. 若 $\pi_{k+1} = \pi_k$ 则停止, 否则回到步骤 2

理论上策略迭代会在有限步内收敛到最优策略 (对于有限 MDP)。但每次策略评估本身需要多轮迭代, 整体开销较大。配套代码 `ch03_dp.py` 实现了完整的策略迭代, 在 FrozenLake 上通常 2-3 次外循环即可收敛。

3.4 值迭代

值迭代 (Value Iteration) 是策略迭代的"懒人版"——不等待策略评估完全收敛就直接改进。它直接对贝尔曼最优方程做迭代:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

值迭代的每次更新都隐式包含了"策略改进", 所以可以直接收敛到 V^* 。收敛后再通过一次策略改进得到 π^* 。值迭代通常比策略迭代快, 因为不需要完整评估。

```
def value_iteration(env, gamma=0.99, theta=1e-8):
    nS, nA = env.observation_space.n, env.action_space.n
    V = np.zeros(nS)
    while True:
        delta = 0
        for s in range(nS):
            q = np.zeros(nA)
            for a in range(nA):
                for (p, s_next, r, done) in env.unwrapped.P[s][a]:
                    q[a] += p * (r + gamma * V[s_next] * (1 - done))
            v_new = q.max()
            delta = max(delta, abs(v_new - V[s]))
            V[s] = v_new
        if delta < theta:
            break
    return V
```

3.5 DP 的局限与现代意义

动态规划需要完全已知 MDP, 且需要遍历所有状态-动作对, 复杂度 $O(|S|^2|A|)$ 。在状态空间巨大 (如 Atari 像素状态) 或连续的情况下, DP 直接失效。但 DP 的思想——"贝尔曼备份"——是所有现代 RL 算法的核心:

- Q-Learning / DQN 是"用样本估计"的值迭代
- MC 是"用完整回合回报"的策略评估
- TD(λ) 是"在 DP 和 MC 之间插值"的方法

理解了 DP, 再看后续算法就豁然开朗——它们都在解决同一个问题: 如何在不知道 MDP 的情况下做 DP 能做的事。

[技巧] 运行 `ch03_dp.py` 后可以看到，策略迭代和值迭代在 FrozenLake 上都能达到 ~75% 的成功率。剩下的失败主要是冰面湿滑导致的随机性——即使最优策略也无法保证 100% 成功。

第四章 蒙特卡洛方法

蒙特卡洛 (Monte Carlo, MC) 方法是"无模型强化学习"的入门算法。它的核心思想非常朴素：通过多次完整回合的经验，用样本平均估计价值函数。MC 不需要 MDP 模型，只要求环境是"分回合"的 (episodic)，且能等到回合结束计算总回报。

4.1 MC 预测

MC 预测 (Prediction) 解决：给定策略 π ，估计 V^π 或 Q^π 。方法是反复运行 π ，记录每个回合的轨迹，然后对每个状态 s 用出现过的回报取平均。根据何时取平均，分为两种变体：

- **首次访问 MC (First-visit MC)**：每个回合中只对 s 的第一次出现计算回报并平均。
- **每次访问 MC (Every-visit MC)**：对 s 的每次出现都计算回报并平均。

两者在无限样本下都收敛到 $V^\pi(s)$ ，但首次访问在数学上更易分析，每次访问在实践中更数据高效。两者的差异在大多数实际问题中可忽略。

4.2 MC 控制

MC 控制 (Control) 的目标是学最优策略。类比策略迭代，需要"评估 + 改进"循环，但这里评估是 MC 估计而非精确 DP。一个关键问题：要估计 $Q^\pi(s,a)$ ，必须确保所有 (s,a) 对都被访问过——这就是"探索"问题。

4.2.1 探索策略： ϵ -greedy

最简单的探索策略是 ϵ -greedy：以 $1-\epsilon$ 概率选择贪心动作 $\arg\max_a Q(s,a)$ ，以 ϵ 概率随机选择动作。这保证所有动作都有非零被选概率，从而确保 (s,a) 对被访问。 ϵ 通常取 0.1，也可以从 1.0 衰减到 0.05（前期多探索，后期多利用）。

4.2.2 on-policy MC 控制算法

完整的 on-policy MC 控制流程如下：

- 1. 初始化 $Q(s,a)$ 任意值， ϵ -soft 策略 π
- 2. 循环：生成一个回合（按 π ）
- 3. 对回合中每个 (s,a) 首次出现，计算 G （从该时刻到回合结束的回报）
- 4. $Q(s,a) \leftarrow Q(s,a) + \alpha(G - Q(s,a))$ （增量平均）
- 5. $\pi(s) \leftarrow \epsilon\text{-greedy}(Q(s,\cdot))$

4.3 Blackjack 实战

Blackjack (21 点) 是 MC 的经典教学环境: 玩家根据自己手牌总和与庄家明牌决定"要牌"或"停牌"。该环境是分回合的, 回报只在回合末确定 (+1 赢 / 0 平 / -1 输), 非常适合 MC——因为中间状态没有有意义的即时奖励。

配套代码 `ch04_mc_control.py` 实现了完整的 MC 控制, 50000 回合训练后, 玩家胜率约 41-43%, 已经接近理论最优 (庄家有先手优势, 玩家最优胜率约 43.5%)。观察学到的策略可以发现一些符合直觉的规律:

- 当玩家点数 ≥ 18 时总是停牌
- 当玩家点数 ≤ 11 时总是要牌 (不会爆)
- 中间区域根据庄家明牌调整: 庄家明牌大时更激进, 庄家明牌小时更保守

4.4 MC 的优缺点

MC 方法的优点是直观、无偏、无需 MDP 模型; 缺点也很明显:

- 必须等回合结束才能更新, 无法在线学习
- 对每个状态的估计方差大 (依赖单次完整回报)
- 不适用于非分回合任务 (如连续控制)

下一章的时序差分 (TD) 方法正是为了解决这些问题而生——它结合了 DP 的单步更新与 MC 的无模型特性, 是现代 RL 的基石。

第五章 时序差分学习: SARSA 与 Q-Learning

时序差分 (Temporal Difference, TD) 学习是强化学习中最重要算法族之一, 它结合了蒙特卡洛的"无模型"特性与动态规划的"单步更新"特性。TD 算法不需要等回合结束, 每一步就能更新, 是现代 RL 应用最广的方法。本章将介绍 TD(0)、SARSA、Q-Learning 三种核心算法, 并通过经典的 CliffWalking 例子对比 on-policy 与 off-policy。

5.1 TD(0) 预测

TD(0) 是最简单的 TD 算法, 用于策略评估。给定策略 π , 用以下更新估计 V^π :

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

方括号内的 $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ 被称为**TD 误差** (TD error), 记作 δ_t 。直观上, TD 用"实际奖励 + 下一状态估值"作为更新目标, 比 MC 的"完整回报"方差小很多, 但有偏差 (自举 bootstrap)。这种"用估计更新估计"的方式看似有循环论证之嫌, 但理论上仍收敛——这是 TD 的妙处。

5.2 SARSA: on-policy TD 控制

SARSA 的名字来源于更新元组 (S, A, R, S', A')——这五个量构成一个完整转移。SARSA 是 on-policy 算法: 用当前策略 π 生成 (s, a, r, s', a'), 然后用这个转移更新 π 自己。其 Q 更新规则为:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

注意 a' 是由当前策略 π 在 s' 处采样的（通常 ϵ -greedy），所以 SARSA 学到的是“考虑探索的策略价值”。这意味着 SARSA 在悬崖边会“保守”——因为它知道未来可能因 ϵ 探索而掉下去，所以会主动远离悬崖。

```
# SARSA 核心循环
state, _ = env.reset()
action = epsilon_greedy(Q, state, nA, epsilon)
while not done:
    next_state, reward, terminated, truncated, _ = env.step(action)
    next_action = epsilon_greedy(Q, next_state, nA, epsilon)
    td_target = reward + gamma * Q[next_state, next_action] * (1 - terminated)
    Q[state, action] += alpha * (td_target - Q[state, action])
    state, action = next_state, next_action
```

5.3 Q-Learning: off-policy TD 控制

Q-Learning 是 RL 历史上最重要的算法之一，Watkins 1989 年提出。与 SARSA 不同，Q-Learning 直接学习最优 Q^* ，与采样策略无关：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

注意更新目标用的是 $\max_{a'} Q(s',a')$ ，而不是 $Q(s',a')$ （实际采样的动作）。这使得 Q-Learning 是 off-policy：可以用任意行为策略（如 ϵ -greedy）收集数据，但更新的目标是“最优策略”。在悬崖边，Q-Learning 假设“未来不会再探索”，所以会贴边走最优路径——但若实际执行 ϵ -greedy，会因偶尔探索而掉下悬崖。

5.4 经典对比：CliffWalking

Sutton & Barto 教科书中的 CliffWalking 是对比 SARSA 与 Q-Learning 的最佳例子。环境是一个 4×12 网格，左下是起点，右下是目标，最下面一行是“悬崖”——掉下去得 -100 并回到起点。普通移动每步 -1，所以最短路径（贴边走）总回报 -13；保守路径（绕开悬崖）约 -25 至 -50。

运行 `ch05_td_sarsa_q.py` 可以观察到经典现象：

- **SARSA** 学到保守路径——绕开悬崖走，在线性能更稳定（约 -25）。
- **Q-Learning** 学到最优路径——贴边走，但训练期因 ϵ 探索常掉悬崖（约 -50）。

这个对比揭示了 on-policy 与 off-policy 的本质差异：on-policy 学“我会怎么走”（考虑探索），off-policy 学“最优怎么走”（不考虑探索）。在实践中，off-policy 数据效率更高（数据可重复使用），但训练可能不稳定。

5.5 期望 SARSA 与算法选择

一个 SARSA 的变体是**期望 SARSA**（Expected SARSA）：用期望替代采样：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma E_{a' \sim \pi} [Q(s',a')] - Q(s,a)]$$

由于消除了 a' 的采样随机性，期望 SARSA 比 SARSA 方差更小。当动作空间小时（如 4 个），计算期望的代价可忽略；动作空间大时则成本较高。

[技巧] 选择建议：若环境回报方差大、训练不稳定 → 优先 SARSA 或 Expected SARSA；若数据有限、需要 off-policy 重用 → 优先 Q-Learning（及后续的 DQN 系列）。

5.6 TD(λ): 在 MC 与 TD 之间插值

TD(0) 只用一步回报（即 $r + \gamma V(s')$ ），MC 用完整回合回报。两者之间可以用 n-step return 平衡：

$$G_{(n)}_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{(n-1)} r_{t+n} + \gamma^n V(s_{t+n})$$

当 $n=1$ 时退化为 TD(0)， $n \rightarrow \infty$ 时退化为 MC。TD(λ) 进一步用几何加权平均所有 n-step return，得到一个统一的算法。实践中 $n=3$ 到 $n=5$ 的多步回报常被使用，能在偏差与方差间取得良好平衡。现代 RL 算法如 Rainbow DQN 中的 N-step Returns 组件就是这种思路。

第六章 函数逼近与 DQN

前几章的表格法（Tabular Method）假设状态空间有限，可以用一个表格存储每个 (s, a) 的 Q 值。但现实问题状态空间往往巨大（如 Atari 像素状态 $84 \times 84 \times 4 \approx 10^5$ 维），甚至连续不可数。函数逼近（Function Approximation）用参数化函数 $Q_\theta(s, a)$ 替代表格，让 RL 第一次有能力处理实际问题。DQN（Deep Q-Network）是 DeepMind 2013 年提出的里程碑算法，首次证明深度神经网络可作为 Q 函数逼近器在复杂任务上稳定训练。

6.1 函数逼近基础

函数逼近的核心是用参数 θ 近似价值函数： $V_\theta(s) \approx V^\pi(s)$ 或 $Q_\theta(s, a) \approx Q^\pi(s, a)$ 。 θ 可以是线性特征模型的权重，也可以是深度神经网络的参数。学习目标是 최소화 均方误差：

$$J(\theta) = E[(G_t - Q_\theta(s_t, a_t))^2]$$

梯度下降更新为： $\theta \leftarrow \theta - \alpha \nabla J(\theta) \approx \theta + \alpha (G_t - Q_\theta(s_t, a_t)) \nabla Q_\theta(s_t, a_t)$ 。若用 TD 目标 $r + \gamma Q(s', a')$ 替代 G_t ，得到“半梯度”TD 算法——之所以叫半梯度，是因为忽略了目标值 $Q(s', a')$ 对 θ 的依赖。在 on-policy 下这种方法收敛，但 off-policy 下可能发散——这是 DQN 引入目标网络的原因。

6.2 DQN 的三大创新

深度神经网络作为 Q 函数看似自然，但 2013 年之前没人能让它稳定训练。问题在于：神经网络的输入是强相关的连续状态（同一回合的相邻帧），且目标值 $Q(s', a')$ 在每次更新后立刻变化（“追逐自己的尾巴”），导致训练严重不稳定。DeepMind 的 DQN 论文用三大创新解决了这些难题：

6.2.1 经验回放 (Experience Replay)

将每一步的 (s, a, r, s', done) 存入回放缓冲区，训练时随机采样 mini-batch。这一招一举三得：

- 打破数据时间相关性 (i.i.d. 假设)
- 提高数据利用率 (每个样本可被多次学习)
- 稳定训练 (避免最近经验过拟合)

回放缓冲区通常大小为 10^4 – 10^6 ，采用 FIFO 策略。

6.2.2 目标网络 (Target Network)

DQN 维护两个网络：策略网络 Q_θ (在线学习) 与目标网络 $Q_{\theta'}$ (计算 TD 目标)。每 N 步 (通常 $N=10$ – 1000) 将 θ 同步到 θ' 。这避免了“目标值跟着 θ 立刻变化”的不稳定——目标值在两次同步之间是固定的。数学上，DQN 的损失函数为：

$$L(\theta) = E[(r + \gamma \max_a Q_{\theta'}(s', a') - Q_\theta(s, a))^2]$$

6.2.3 ϵ -greedy 探索 + 帧堆叠

DQN 用 ϵ -greedy 探索 (ϵ 从 1.0 衰减到 0.05)，并采用 4 帧堆叠作为状态 (让网络感知速度)。这些工程细节看似简单，但对最终性能至关重要——许多复现失败都是因忽略了这些细节。

6.3 DQN 完整算法

下面是 DQN 的伪代码，配套代码 `ch06_dqn.py` 是完整 PyTorch 实现。

```
# DQN 主循环
for episode in range(num_episodes):
    state = env.reset()
    for t in range(max_steps):
        action = agent.act(state) #  $\epsilon$ -greedy
        next_state, reward, done, _ = env.step(action)
        buffer.push(state, action, reward, next_state, done)
        state = next_state
    agent.learn() # 从 buffer 采样训练
    if episode % target_sync == 0:
        agent.sync_target() # 同步目标网络
```

其中 `agent.learn()` 的核心代码如下：

```
def learn(self):
    batch = buffer.sample(64)
    s, a, r, s_next, done = batch
    q_sa = policy_net(s).gather(1, a).squeeze() # Q(s,a)
    with torch.no_grad():
        q_next = target_net(s_next).max(dim=1)[0] # max_a' Q(s',a')
    q_target = r + gamma * q_next * (1 - done) # TD target
    loss = mse_loss(q_sa, q_target)
    optimizer.zero_grad()
    loss.backward()
    clip_grad_norm_(policy_net.parameters(), 10.0) # 梯度裁剪
    optimizer.step()
```

6.4 训练技巧与超参数

DQN 在 CartPole-v1 上的训练并不困难，但要达到 500 分稳定需要调参。下表是经过实践验证的推荐超参数：

超参数	推荐值	说明
学习率 lr	1e-3	Adam 优化器；过高训练不稳，过低收敛慢
折扣因子 γ	0.99	CartPole 几乎无随机性，可接近 1
回放缓冲区大小	10,000	CartPole 状态简单；Atari 需 1e5-1e6
批量大小	64	越大越稳但越慢；常用 32-256
目标网络同步	每 10 episodes	或用软更新 $\tau=0.005$
ϵ 衰减	$\exp(-\text{steps}/500)$	从 1.0 衰减到 0.05
梯度裁剪	10.0	防止 Q 值爆炸
奖励塑形	终止罚 -10	CartPole 经验技巧

表 6.1: DQN 在 CartPole-v1 上的推荐超参数

6.5 DQN 的局限

尽管 DQN 是巨大突破，但它仍有几个明显问题：

- **Q 值过估**：max 操作导致系统性高估 Q 值（解决：Double DQN，第十一章）
- **无法处理连续动作**：max over a 在连续空间不可行（解决：DDPG/SAC，第十章）
- **样本效率低**：均匀采样浪费了高价值样本（解决：优先经验回放 PER）
- **价值与优势不分离**：Q 同时编码"状态好坏"与"动作优势"（解决：Dueling DQN）

这些局限在后续章节中会被一一解决。DQN 的真正价值不仅是"第一个能用的深度 RL 算法"，更是为后续 Rainbow、SAC 等算法提供了基础架构。

第七章 策略梯度方法：REINFORCE

基于值的方法（DQN）通过学习 Q^* 间接得到策略，但有以下局限：难以处理连续动作空间（ $\max$ 不可微）；随机策略无法表达（ $\arg\max$ 总是确定性）；在部分可观察环境中性能受限。基于策略的方法直接参数化 $\pi_\theta(a|s)$ ，避免了这些限制。本章介绍策略梯度定理的推导与最简单的策略梯度算法 REINFORCE。

7.1 策略目标函数

策略梯度直接优化策略参数 θ ，使其在某种意义下"更好"。常用的目标函数有三种：

- **episodic 情形**: $J(\theta) = E_{\pi_\theta} [G_0]$ （起始状态价值）
- **continuing 情形**: $J(\theta) = \text{平均奖励}$ （每步期望回报）
- **discounted 情形**: $J(\theta) = E[\sum \gamma^t r_t]$

无论哪种，目标都是最大化 $J(\theta)$ 。策略梯度方法的核心是计算 $\nabla_\theta J(\theta)$ ，然后用梯度上升更新 θ 。

7.2 策略梯度定理

策略梯度定理（Policy Gradient Theorem）是 RL 理论中最重要的结论之一。它告诉我们：无论状态分布如何，目标函数的梯度都可以写成期望形式：

$$\nabla_\theta J(\theta) = E_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s,a)]$$

这个结论的奇妙之处在于： $\nabla_\theta J(\theta)$ 不需要对状态分布求导（状态分布依赖 θ ，求导复杂）。它将"优化目标"转化为"在当前策略下采样 (s,a) ，按 $\nabla \log \pi \cdot Q$ 加权求和"。**直观理解**：若某动作的 Q 较高（"好"），就增大 $\log \pi(a|s)$ 的梯度（让 π 更倾向选它）；反之则减小。这正是"好动作多出现，坏动作少出现"的本质。

7.3 REINFORCE 算法

REINFORCE（Williams 1992）是最简单的策略梯度算法：用实际回报 G_t 替代 $Q^{\pi}(s,a)$ ：

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

算法流程：

- 1. 用当前策略 π_θ 生成一个完整回合
- 2. 对每个 (s_t, a_t) ，计算 G_t （从 t 到回合结束的回报）
- 3. 累加梯度 $\nabla \log \pi(a_t|s_t) \cdot G_t$
- 4. 回合结束后用累计梯度更新 θ

```
# REINFORCE 核心 PyTorch 实现
log_probs, rewards = [], []
for t in range(T):
    logits = policy(state)
    dist = Categorical(logits=logits)
    action = dist.sample()
    log_probs.append(dist.log_prob(action))
    state, reward, done, _ = env.step(action.item())
    rewards.append(reward)

# 计算回报
returns = []
G = 0
for r in reversed(rewards):
    G = gamma * G + r
    returns.insert(0, G)

# 策略梯度更新
loss = -(torch.stack(log_probs) * torch.tensor(returns)).sum()
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

7.4 减小方差：基线与优势

REINFORCE 的最大问题是方差极大——单个回合的 G_t 可能从 -100 到 +1000，导致梯度估计噪声极大，训练极慢甚至不收敛。一个简单但有效的改进是引入**基线**（Baseline）：

$$\nabla_{\theta} J(\theta) = E[\nabla \log \pi(a|s) \cdot (Q(s,a) - b(s))]$$

其中 $b(s)$ 是任意不依赖 a 的函数（不影响期望），最常用的是 $V(s)$ 。此时 $Q(s,a) - V(s)$ 被称为**优势函数** $A(s,a) = Q(s,a) - V(s)$ 。直观上： $A > 0$ 表示该动作比平均好，应增加概率； $A < 0$ 表示比平均差，应减少概率。用 A 替代 Q 大幅减小方差，因为去除了“状态本身好坏”的常数部分。

$V(s)$ 可以用另一个网络（Critic）学习，这就引出了下一章的 Actor-Critic 框架。另一个常见技巧是**回报标准化**：将 G 标准化为 $(G - \text{mean}) / (\text{std} + \epsilon)$ ，消除不同回合间奖励尺度差异。

7.5 REINFORCE 的局限与改进方向

REINFORCE 的几个明显缺陷：

- 必须等回合结束才能更新（不适用于连续任务）
- 方差大（即使加 baseline 也比 TD 方法大）
- 样本效率低（on-policy，每个样本只用一次）

解决这些问题的方式分别对应后续章节：用 TD 替代 MC 回报 → Actor-Critic（第八章）；用多个 step 累积更新 → A3C；用重要性采样重复利用数据 → PPO（第九章）。策略梯度定理是这些现代算法的共同基础。

第八章 Actor-Critic 与 A2C/A3C

Actor-Critic (AC) 框架是策略梯度与值函数方法的集大成者。"Actor" 是策略网络 π_{θ} , "Critic" 是价值网络 V_{ϕ} 。Actor 用 Critic 估计的优势更新策略, Critic 用 TD 误差更新价值。这种"角色分工"既保留了策略梯度的灵活性(可处理连续动作、随机策略), 又获得了 TD 方法的小方差与在线学习优势。

8.1 AC 框架的核心思想

在 REINFORCE 中, 我们用实际回报 G_t 作为 Q 的估计; 在 AC 中, 用 TD 误差 δ_t 作为优势 A 的估计:

$$\delta_t = r_{t+1} + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

$$A(s_t, a_t) \approx \delta_t$$

这是因为 $E[\delta_t] = E[Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)] = A^{\pi}(s_t, a_t)$, 所以 δ 是 A 的无偏估计, 但方差远小于 G_t 。更新规则变为:

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) \cdot \delta_t$$

$$\phi \leftarrow \phi + \beta \delta_t \nabla V_{\phi}(s_t)$$

8.2 A2C: 同步 Actor-Critic

A2C (Advantage Actor-Critic) 是 AC 的标准实现。关键细节:

- Actor 与 Critic 共享底层网络(部分参数), 输出端分头(policy head + value head)
- 用多步累积(rollout)代替单步 TD, 进一步降低方差
- 加入熵正则化(entropy bonus)鼓励探索
- 总损失 = actor_loss + 0.5 * critic_loss + 0.01 * entropy_loss

```
# A2C 总损失
actor_loss = -(log_probs * advantages.detach()).mean()
critic_loss = mse_loss(values, returns)
entropy_loss = -dist.entropy().mean() # 鼓励探索
total_loss = actor_loss + 0.5 * critic_loss + 0.01 * entropy_loss
```

熵正则化非常重要: 没有它, 策略会快速收敛到确定性策略而停止探索; 有它则保持一定随机性, 避免陷入局部最优。系数 0.01 是常用值。

8.3 A3C: 异步并行

A3C (Asynchronous Advantage Actor-Critic) 是 DeepMind 2016 年的工作, 与 A2C 的算法完全相同, 区别在于"异步并行":

- 多个 worker 进程同时与环境交互, 各自维护一份策略副本
- worker 定期将梯度推送到全局网络, 并从全局网络同步参数
- 不同 worker 经历不同状态, 相当于"内在探索"

异步并行的副作用是消除数据相关性(不同 worker 的样本是 i.i.d. 的), 类似于 DQN 的经验回放, 但不需要缓冲区。A3C 在 Atari 上首次达到了与 DQN 相当的性能, 但训练速度快

数倍。

8.4 GAE：广义优势估计

单步优势 δ_t 方差小但有偏差；多步优势 $G_{(n)}_t$ 偏差小但方差大。Schulman 等人 2015 年提出的 GAE (Generalized Advantage Estimation) 在两者间指数加权平均：

$$A_{\text{GAE}}(\gamma, \lambda)_t = \sum_{k=0}^{\infty} (\gamma \lambda)^k \cdot \delta_{t+k}$$

当 $\lambda=0$ 时退化为单步 TD (δ_t)； $\lambda=1$ 时退化为 MC ($G_t - V(s_t)$)。实践中 $\lambda=0.95$ 几乎是黄金值，在偏差与方差间取得最佳平衡。GAE 是 PPO、TRPO 等现代算法的标准组件，几乎所有 on-policy 算法都使用它。

8.5 A2C 实现要点

配套代码 `ch08_a2c.py` 实现了完整的 A2C 算法。几个关键工程细节：

- **网络结构**：输入 $\rightarrow$ 128 Tanh $\rightarrow$ 128 Tanh $\rightarrow$ (policy_head: Linear(action_dim) + value_head: Linear(1))
- **优化器**：Actor 与 Critic 用同一个 Adam 优化器（学习率 $5e-4$ ）
- **rollout**：每 5 步累积一次更新，比单步稳定
- **梯度裁剪**：clip_grad_norm_ 防止爆炸

运行该代码，约 200-400 回合可让 CartPole-v1 达到 500 分。相比 DQN，A2C 训练曲线更平稳（无 target network 的延迟），但样本效率略低（on-policy）。

第九章 PPO：现代 RL 工业标准

PPO (Proximal Policy Optimization, Schulman et al. 2017) 是目前应用最广的 on-policy 深度 RL 算法。它解决了 REINFORCE/A2C 的样本效率问题，又比 TRPO 简单易实现。OpenAI Five、ChatGPT 的 RLHF 阶段都用 PPO，它已成为 RL 的“工业标准”。本章详细推导 PPO 的损失函数、超参数选择与完整实现。

9.1 从 TRPO 到 PPO

TRPO (Trust Region Policy Optimization, 2015) 的核心思想是“信任域”约束：每次更新限制新策略与旧策略的 KL 散度不超过 δ （通常 0.01），保证策略不会“变化过大”导致崩溃。数学上，TRPO 求解：

$$\max_{\theta} E[\pi_{\theta}(a|s)/\pi_{\theta_{\text{old}}}(a|s) \cdot A(s,a)] \text{ subject to } E[\text{KL}(\pi_{\theta_{\text{old}}}||\pi_{\theta})] \leq \delta$$

TRPO 用共轭梯度法求解这个约束优化，实现复杂且计算开销大。PPO 的洞察是：可以用更简单的“裁剪”近似这个约束——这就是 PPO-Clip。

9.2 PPO-Clip 损失函数

PPO 用重要性采样比 $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ 衡量新旧策略差异。当 r 远离 1 时策略变化大，应限制。PPO-Clip 直接在 r 上裁剪：

$$L^{\text{CLIP}}(\theta) = E[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t)]$$

其中 ϵ 通常取 0.2。直观理解：

- 若 $A > 0$ （好动作）：希望增大 r ，但 $r > 1+\epsilon$ 时不再奖励（防止过激进）
- 若 $A < 0$ （坏动作）：希望减小 r ，但 $r < 1-\epsilon$ 时不再惩罚
- 结果：策略变化被限制在 $[1-\epsilon, 1+\epsilon]$ 区间内，等价于软 KL 约束

9.3 PPO 完整目标

PPO 的总损失结合了策略损失、价值损失、熵正则三项：

$$L_{\text{PPO}} = L_{\text{CLIP}} - c_1 \cdot L_{\text{VF}} + c_2 \cdot S[\pi_\theta]$$

其中：

- $L_{\text{VF}} = (V_\theta(s) - V_{\text{target}})^2$ ，价值损失（也常加裁剪）
- $S[\pi_\theta]$ 是策略熵，鼓励探索
- $c_1 = 0.5$, $c_2 = 0.01$ 是经验值

9.4 PPO 训练流程

PPO 的训练流程与 A2C 类似，但有几个关键差异：

- 1. 用 $\pi_{\theta_{\text{old}}}$ 收集一批数据（通常 2048 步）
- 2. 计算 GAE 优势与回报
- 3. **在同一批数据上做多次 epoch 更新**（通常 4-10 个 epoch）
- 4. 每个 epoch 内分 mini-batch（如 64 样本）更新
- 5. 更新后 $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ ，开始下一轮收集

第 3 步是 PPO 与 A2C 的本质区别：A2C 每批数据只用一次就丢弃，PPO 重复利用多次。裁剪机制保证了"重复利用"不会导致策略偏离数据生成分布太远。

```
# PPO 核心更新
for epoch in range(4): # 多 epoch
    for mb_idx in mini_batches:
        # 计算新策略的 log_prob
        logits = policy(states[mb_idx])
        new_log_prob = Categorical(logits=logits).log_prob(actions[mb_idx])
        ratio = (new_log_prob - old_log_probs[mb_idx]).exp()

        # clipped surrogate
        surr1 = ratio * advantages[mb_idx]
        surr2 = torch.clamp(ratio, 1-eps, 1+eps) * advantages[mb_idx]
        actor_loss = -torch.min(surr1, surr2).mean()

        # value loss (also clipped)
        v = critic(states[mb_idx])
        v_clipped = returns[mb_idx] + torch.clamp(v - returns[mb_idx], -eps, eps)
        value_loss = torch.max((v - returns[mb_idx])**2,
                                (v_clipped - returns[mb_idx])**2).mean()

    loss = actor_loss + 0.5 * value_loss - 0.01 * entropy
```

9.5 超参数选择

超参数	推荐值	说明
学习率	3e-4	Adam; 线性衰减到 0 可提升后期稳定
clip ϵ	0.2	过大不稳, 过小学不到
epochs/batch	4	Atari 大状态可 10
mini-batch size	64	通常 32-256
steps per iter	2048	越大越稳但越慢
γ	0.99	默认
GAE λ	0.95	偏差-方差平衡
entropy coef	0.01	Atari 可 0.005
value coef	0.5	常用

表 9.1: PPO 推荐超参数 (CartPole/MuJoCo 通用)

9.6 PPO 的工程实践

- 配套代码 `ch09_ppo.py` 是完整实现。几个工程要点:
- **obs 标准化**: 维护运行均值与方差, 对 obs 做 $(x - \text{mean}) / \text{std}$ 。MuJoCo 必备, CartPole 可选
 - **reward scaling**: 除以运行 std, 避免奖励幅度差异
 - **优势标准化**: 每批 $\text{adv} = (\text{adv} - \text{mean}) / (\text{std} + 1e-8)$
 - **学习率衰减**: 后期降低 lr 可进一步提升

· **正交初始化**：网络权重用 `nn.init.orthogonal_`，提升稳定性

[技巧] PPO 在 CartPole 上 100-200 个 iteration（约 200K 步）可稳定达到 500 分。
在 MuJoCo 上需 1M-10M 步，建议用 Gymnasium 的 vectorized env 加速。

第十章 SAC 与连续控制

Soft Actor-Critic (SAC, Haarnoja et al. 2018) 是目前连续控制任务上的 SOTA 算法。它基于"最大熵 RL"框架，在期望回报之外还优化策略熵——使策略既成功又随机。这种"随机性"不仅是探索手段，更让策略在不确定性下更鲁棒。SAC 在 MuJoCo、Pendulum、机器人控制等连续任务上全面超过 PPO 与 DDPG。

10.1 最大熵 RL

标准 RL 的目标是最大化期望回报 $E[\sum \gamma^t r_t]$ 。最大熵 RL 在此基础上加上熵奖励：

$$J(\pi) = \sum_t E[(r_t + \alpha H(\pi(\cdot|s_t)))]$$

其中 H 是策略熵， α 是温度系数控制熵权重。直观上：智能体不仅想获得高回报，还想"尽可能随机地"获得高回报——这种随机性带来更强的探索能力和对环境变化的鲁棒性。最大熵 RL 的 Bellman 方程变为：

$$Q^\pi(s,a) = E[r + \gamma (Q^\pi(s',a') - \alpha \log \pi(a'|s'))]$$

10.2 SAC 三大创新

10.2.1 双 Q 网络 (Clipped Double Q)

类似 TD3，SAC 用两个独立的 Q 网络 Q_1, Q_2 取最小值，抑制 Q 值过估问题（连续控制中过估问题比离散更严重）。目标值计算： $Q_target = r + \gamma (\min(Q_1', Q_2') - \alpha \log \pi(a'|s'))$ 。

10.2.2 自动温度系数 α

α 的选择很难：太大策略太随机，太小探索不足。SAC 引入"自动 α "——让熵接近目标熵（通常 $-\dim(A)$ ，如连续 3 维动作 -3）：

$$J(\alpha) = E[-\alpha \log \pi(a|s) - \alpha H_{target}]$$

α 作为可学习参数，用梯度下降更新。训练初期熵高时 α 减小（少奖励熵），训练后期熵低时 α 增大（多奖励熵，鼓励探索）。

10.2.3 重参数化技巧 (Reparameterization Trick)

SAC 的策略是高斯分布 $N(\mu(s), \sigma(s))$ 。直接采样 $a \sim N(\mu, \sigma)$ 不可微，无法将梯度传回策略网络。重参数化技巧： $a = \mu + \sigma \cdot \epsilon$ ，其中 $\epsilon \sim N(0,1)$ 。这样 a 关于 (μ, σ) 可微，梯度可通过 a 流回策略网络。

10.3 SAC 完整算法

下面是 SAC 的核心伪代码：

```
# SAC 主循环
for step in range(num_steps):
    a, log_pi = policy.sample(state) # 重参数化采样
    next_state, r, done = env.step(a)
    buffer.push(state, a, r, next_state, done)
    state = next_state

# 更新 Q
batch = buffer.sample(256)
with torch.no_grad():
    a_next, log_pi_next = policy.sample(s_next)
    q_target = r + gamma * (min(Q1'(s_next, a_next),
                                Q2'(s_next, a_next))
                        - alpha * log_pi_next)
    q1_loss = mse(Q1(s, a), q_target)
    q2_loss = mse(Q2(s, a), q_target)

# 更新 policy
a_new, log_pi_new = policy.sample(s) # 重参数化
q_new = min(Q1(s, a_new), Q2(s, a_new))
policy_loss = (alpha * log_pi_new - q_new).mean()

# 更新 alpha (自动)
alpha_loss = -(log_alpha * (log_pi_new + target_entropy).detach()).mean()

# 软更新 target Q
soft_update(Q1_target, Q1, tau=0.005)
soft_update(Q2_target, Q2, tau=0.005)
```

10.4 SAC 在 Pendulum 上的表现

配套代码 `ch10_sac.py` 在 Pendulum-v1 上训练。Pendulum 是经典连续控制任务：一根杆子，可以用 $[-2, 2]$ 范围的力矩使其立起。奖励是 $-\theta^2 - 0.1 \cdot (\text{action})^2$ ，最大约 0，最差约 -16。SAC 通常 20K 步内可将平均奖励从 -1600 提升到 -200 左右，比 DDPG/TD3 更稳。

10.5 SAC vs PPO vs DDPG/TD3

算法	on/off	动作空间	样本效率	稳定性	适用场景
PPO	on-policy	离散/连续	低	高	通用，并行训练
SAC	off-policy	连续	高	高	连续控制首选
TD3	off-policy	连续	高	中	比 DDPG 稳定
DDPG	off-policy	连续	高	低	已淘汰
DQN	off-policy	离散	中	中	离散控制

表 10.1：主流深度 RL 算法对比

[技巧] SAC 的优势在样本效率：在 MuJoCo HalfCheetah 上，SAC 100K 步可达 DDPG 1M 步的效果。若你的任务环境模拟代价高（如真实机器人），SAC 是不二之选。

第十一章 Rainbow DQN 与改进技巧

Rainbow DQN (Hessel et al. 2018) 整合了 DQN 的六项改进，在 Atari 57 游戏上大幅超越原始 DQN，是值方法的高峰。本章介绍其中三项最重要的改进——Double DQN、Dueling Network、Prioritized Experience Replay，并简要介绍其余三项。

11.1 Double DQN：解决 Q 值过估

原始 DQN 的目标值是 $r + \gamma \max_a Q_{\text{target}}(s', a)$ ，其中 $\max$ 操作会"放大"Q 估计的误差——若 Q 对某个动作高估， $\max$ 会选中这个动作，进一步强化高估。van Hasselt 等人 2015 年提出 Double DQN：把"选动作"和"估价"分开：

$$Q_{\text{target}} = r + \gamma Q_{\text{target}}(s', \operatorname{argmax}_a Q_{\text{policy}}(s', a))$$

用 `policy_net` 选动作（"我认为哪个动作好"），用 `target_net` 估价（"客观上值多少"）。由于两个网络独立训练，过估误差被打破相关性，显著降低。实现只需改动一行代码：

```
# 原始 DQN
q_next = target_net(s_next).max(dim=1)[0]

# Double DQN
a_next = policy_net(s_next).argmax(dim=1, keepdim=True)
q_next = target_net(s_next).gather(1, a_next).squeeze(1)
```

11.2 Dueling Network：分离 V 与 A

Dueling DQN (Wang et al. 2015) 改变了网络结构：将 $Q(s, a)$ 分解为 $V(s) + A(s, a)$ ，其中 $V(s)$ 评估"状态本身的好坏"， $A(s, a)$ 评估"在状态 s 下动作 a 相对平均的优势"。数学上 $Q(s, a) = V(s) + A(s, a) - \text{mean}(A(s, \cdot))$ （减均值保证可识别性）。这种分解让网络能更高效地学习——许多状态下动作选择无关紧要（如已经有杆子稳定平衡），此时只需更新 V 而 A 保持小值，避免无意义的参数更新。

```
class DuelingQNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.feature = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU())
        self.value = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, 1)) # V(s)
        self.advantage = nn.Sequential(
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, action_dim)) # A(s,a)

    def forward(self, x):
        h = self.feature(x)
        v = self.value(h)
        a = self.advantage(h)
        return v + (a - a.mean(dim=1, keepdim=True))
```

11.3 Prioritized Experience Replay

均匀采样浪费了"高价值"样本——那些 TD 误差大的转移蕴含更多学习信号。PER (Schaul et al. 2015) 按 |TD 误差| 比例采样：

$$P(i) = p_i^\alpha / \sum_k p_k^\alpha, \text{ 其中 } p_i = |\delta_i| + \epsilon$$

α 控制优先化程度 ($\alpha=0$ 退化为均匀, $\alpha=1$ 完全优先)。但优先采样改变了数据分布, 需用重要性采样权重修正：

$$w_i = (1/(N \cdot P(i)))^\beta$$

β 从 0.4 线性退火到 1.0 (最终完全无偏)。PER 实现稍复杂, 但效果显著——尤其在稀疏奖励任务中。

11.4 其他三项改进

完整 Rainbow 还包含：

- **Multi-step Returns**: 用 n-step TD 替代 1-step, 平衡偏差与方差 (n=3 常用)
- **Distributional RL (C51)**: 学 Q 值的分布而非期望, 更稳定
- **NoisyNet**: 在网络中加入可学习噪声参数, 替代 ϵ -greedy 探索

这六项改进在 Atari 上每一项都有显著提升, 组合后达到 SOTA。实践中不一定全部使用——Double + Dueling + PER 是性价比最高的组合。

11.5 实战与对比

配套代码 ch11_rainbow.py 实现了 Double + Dueling + PER 三项组合。运行后对比原始 DQN (ch06_dqn.py), 可以观察到：

- 训练曲线更平稳 (Double 抑制过估)
- 收敛速度更快 20-30% (Dueling 提高学习效率)

· 早期学习更高效（PER 聚焦高信号样本）

改进项	效果	实现复杂度	推荐
原始 DQN	基线	低	教学
+ Double	抑制过估，稳定训练	低（改 1 行）	强烈推荐
+ Dueling	收敛更快	中（改网络）	推荐
+ PER	样本效率 +20-30%	中（改 buffer）	推荐
+ Multi-step	方差-偏差平衡	低（改 target）	可选
+ C51	更稳定但复杂	高	研究用
+ NoisyNet	替代 ϵ -greedy	中	可选
= Rainbow	SOTA on Atari	高	生产

表 11.1: DQN 改进项的性价比分析

第十二章 前沿主题：RLHF、世界模型、离线 RL、MARL

本章覆盖 2020 年后强化学习领域的四个热门方向。这些主题正在迅速发展，已成为顶级会议的核心议题。我们将介绍每个方向的核心思想、代表性算法与开源资源，为读者进一步研究提供起点。

12.1 RLHF：驱动大模型对齐

RLHF（Reinforcement Learning from Human Feedback）是 ChatGPT、Claude、GPT-4 等大模型对齐人类偏好的关键技术。其流程分三阶段：

- **阶段一：SFT（Supervised Fine-Tuning）**：用高质量指令数据微调基座模型，让其学会"按指令生成"
 - **阶段二：奖励模型 RM 训练**：收集人类对模型输出的偏好对 (y_w, y_l) ，训练一个 RM 模型，使其输出 $r(x,y)$ 反映人类偏好
 - **阶段三：PPO 微调**：用 RM 作为奖励，PPO 优化 LLM 策略，同时加 KL 惩罚防止 LLM 偏离 SFT 模型太远
- RLHF 的关键损失函数：

$$L_{\text{RLHF}} = E[r_{\text{RM}}(x, y)] - \beta \cdot \text{KL}[\pi_{\theta} || \pi_{\text{SFT}}]$$

KL 项是 RLHF 的核心防御机制：没有它，LLM 可能"reward hacking"——学会输出 RM 给高分但实际无意义的内容。配套代码 `ch12_rlhf_concept.py` 用 toy 环境演示了这个机制——观察 KL 系数 β 的变化如何影响最终策略。

12.2 世界模型：Dreamer 系列

基于模型 RL 的难点是学准确的环境模型。Dreamer (Hafner et al. 2020) 系列采用"潜在空间世界模型": 在紧凑的潜在空间中学习环境动力学, 再用该模型"想象"经验训练策略。流程:

- 1. 用 VAE 类编码器将图像观测编码到潜在向量 z_t
- 2. 在潜在空间学 RSSM (Recurrent State-Space Model): 预测下一潜在状态
- 3. 在模型中"想象"长轨迹, 用 actor-critic 训练策略

DreamerV3 (2023) 在 150+ 任务上达到 SOTA, 且无需超参数调整——这是基于模型 RL 的重大突破。世界模型的优势: 极低样本消耗 (比无模型少 100 倍), 且模型可迁移到下游任务。

12.3 离线 RL: 从固定数据集学习

传统 RL 假设智能体可在线与环境交互, 但医疗、金融、自动驾驶等领域在线试错代价高昂甚至危险。离线 RL (Offline RL / Batch RL) 从固定数据集 $D = \{(s, a, r, s')\}$ 中学习策略, 无需额外交互。核心挑战是**分布偏移**: 若学到的策略与数据集分布差异大, Q 值估计会发散 (外推误差)。主要算法:

- **CQL (Conservative Q-Learning)**: 在 Q 上加正则项, 惩罚未见过的 (s,a) 对的 Q 值, 使其保守低估
- **IQL (Implicit Q-Learning)**: 用 expectile regression 学 V, 避免查询未见过的动作, 更稳定
- **Decision Transformer**: 把 RL 转化为序列建模, 用 Transformer 直接学 return-to-go $\rightarrow$ action 映射

离线 RL 是 RL 落地真实世界的关键技术, 近两年发展极快。

12.4 多智能体 RL: MARL

当多个智能体共享环境时, 每个智能体的最优策略依赖其他智能体的策略——这就进入了博弈论范畴。MARL (Multi-Agent RL) 的核心概念:

- **完全合作**: 所有智能体共享奖励, 目标一致。如机器人协作搬物。
- **完全竞争**: 零和博弈。如围棋、星际争霸。
- **混合**: 既有合作又有竞争。如 Dota 2 5v5。

主流算法:

- **Independent Q-Learning (IQL)**: 每个智能体独立学习, 最简单但不稳定
- **MADDPG**: 每个智能体学自己的 Q, 但 Q 输入包含所有智能体动作 (centralized training, decentralized execution)
- **MAPPO**: PPO 的多智能体扩展, OpenAI Five、AlphaStar 都用类似方法
- **HATPO**: 基于 transformer 的多智能体, 2022 年 SOTA

12.5 大模型时代的 RL 新方向

2023 年以来，大语言模型 + RL 出现许多新方向：

- **RLAIF**：用 AI（而非人类）提供反馈，替代 RLHF 中的人类标注
- **DPO (Direct Preference Optimization)**：跳过 RM，直接用偏好对优化策略，简化 RLHF 流程
- **Self-Play**：智能体与自己对弈，AlphaGo、AlphaZero 的核心思想
- **In-context RL**：让 LLM 在推理时快速适应新任务，无需重新训练

RL 与大模型的结合正在催生新一轮研究热潮。掌握本教程前 11 章的基础后，读者应能阅读这些前沿论文并跟进最新进展。

第十三章 算法对比与工程实践

学完前 12 章的算法后，本章从工程视角整合：如何为具体任务选择算法、如何分析训练曲线、如何调参、如何复现论文结果。这些"实战经验"是从"会跑代码"到"能解决问题"的关键一跃。

13.1 算法选择决策树

不同算法适用于不同场景，没有"万能算法"。下面的决策树总结了几十年的实践经验：

- **动作空间离散？** 是 → 值方法（DQN 系列）；否 → 策略方法（PPO/SAC）
- **能否并行环境？** 是 → PPO（并行是优势）；否 → SAC（数据效率高）
- **样本代价高？** 是 → 基于模型方法（Dreamer）或离线 RL；否 → 无模型 OK
- **稀疏奖励？** 是 → 加 intrinsic motivation / HER / 课程学习；否 → 标准 OK
- **多智能体？** 是 → MAPPO 或 MADDPG；否 → 单智能体算法
- **需要在线学习？** 是 → PPO（on-policy 适合连续适应）；否 → off-policy 可重用数据

13.2 算法综合对比

算法	类型	动作空间	样本效率	稳定	并行	实现难度	推荐度
Q-Learning	off-policy	离散	高	中	差	低	★★★★
DQN	off-policy	离散	中	中	中	中	★★★★★
Rainbow	off-policy	离散	中高	高	中	高	★★★★★
REINFORCE	on-policy	离散/连续	低	低	好	低	★★
A2C/A3C	on-policy	离散/连续	低	中	好	中	★★★★
PPO	on-policy	离散/连续	低中	高	好	中	★★★★★

算法	类型	动作空间	样本效率	稳定	并行	实现难度	推荐度
DDPG	off-policy	连续	高	低	差	中	★★
TD3	off-policy	连续	高	中	差	中	★★★★ ★
SAC	off-policy	连续	高	高	差	中高	★★★★ ★★

表 13.1: 主流 RL 算法综合对比

13.3 超参数经验

超参数	常见范围	经验法则
学习率 lr	1e-4 ~ 1e-3	Adam; 过大训练震荡, 过小收敛慢
折扣因子 γ	0.9 ~ 0.99	长期收益重要用 0.99; 短视任务可 0.9
批量大小 batch	32 ~ 256	越大越稳但越慢; GPU 显存限制
回放缓冲区	1e4 ~ 1e6	小任务 1e4 够; Atari 需 1e5+
目标网络同步	5 ~ 100 episodes	或用软更新 $\tau=0.005$
ϵ (探索)	0.05 ~ 0.3	从 1.0 指数衰减到 0.05
熵系数	0.001 ~ 0.05	探索不足时增大; Atari 0.01
GAE λ	0.9 ~ 0.97	默认 0.95
clip ϵ (PPO)	0.1 ~ 0.3	默认 0.2
τ (软更新)	0.001 ~ 0.01	SAC 0.005; TD3 0.005

表 13.2: RL 常用超参数经验值

13.4 训练曲线分析

如何判断训练是否成功? 看奖励曲线是第一步, 但还远远不够。一个完整的训练诊断应该包括:

- **奖励曲线**: 是否上升? 最终值? 滑动平均是否稳定?
- **评估曲线**: 定期用 eval_mode (无探索) 评估, 反映真实策略性能
- **损失曲线**: actor_loss / critic_loss / value_loss 是否收敛?
- **Q 值统计**: Q 值是否爆炸? 均值是否合理?
- **熵曲线**: 策略熵是否过早降到 0?
- **学习率**: 是否按计划衰减?

[注意] 一个常见陷阱：奖励上升但 eval 性能下降——这通常是策略过拟合了特定探索路径。解决：用更大 buffer、更小 lr、或更严格的熵正则。

13.5 工程实践要点

13.5.1 复现论文的"潜规则"

RL 论文复现极难——同样的算法、同样的超参数，不同实现性能可能差几倍。主要原因：网络初始化、obs 标准化、reward scaling、梯度裁剪、优化器细节等"工程技巧"经常不在论文中描述。建议复现时：

- 参考 OpenAI baselines / stable-baselines3 / CleanRL 的实现
- 用相同的环境版本（Atari 有 multiple versions）
- 固定随机种子，重复 5+ 次取平均与方差
- 报告多 seed 结果，不只展示"最好"的曲线

13.5.2 推荐代码库

- **CleanRL**：单文件实现，最易学习与修改
- **Stable-Baselines3**：工业化封装，开箱即用
- **RLlib (Ray)**：分布式训练，生产级
- **DI-engine**：国产，中文文档完善
- **JAX 实现**（如 PureJaxRL）：GPU/TPU 加速，研究前沿

13.6 算法对比的可视化思维

最后，给读者一个直观的"算法地图"：

- **离散 vs 连续**：Q-Learning/DQN 处理离散；PPO/SAC 处理两者；DDPG/TD3 只连续
- **On vs Off policy**：PPO 是 on-policy 代表；SAC/DQN 是 off-policy 代表
- **基于值 vs 基于策略**：DQN 学 Q；REINFORCE/PPO 直接学 π ；AC 兼学
- **有模型 vs 无模型**：Dreamer 有模型；其他主流无模型
- **熵正则**：SAC 显式最大化熵；PPO 用熵 bonus；DQN 无显式熵

这个"五维分类"覆盖了 95% 的 RL 算法。读者在阅读新论文时，可以先定位它在每个维度上的位置，就能快速理解其本质。

第十四章 调试、调参与常见问题排查

强化学习训练常被称为"黑暗艺术"——即使算法正确，超参数不对也可能完全不收敛。本章总结作者在实践中遇到的高频问题及解决方案，帮助读者避开常见陷阱。建议在跑每个新算法时，先对照本章检查清单自查一遍。

14.1 训练不收敛：可能的原因

如果训练几百回合后奖励仍不上升，按以下顺序排查：

- **环境 API**：Gymnasium 0.26+ 的 5 元组返回值是否正确处理？reset 返回 (obs, info) 是否解包？
- **奖励范围**：奖励是否合理？是否需要 scaling（除以最大值）？
- **obs 标准化**：连续控制任务必须做；CartPole 等小状态可不
- **学习率**：太大训练震荡，太小不收敛。先试 $1e-3$ ，不行降到 $1e-4$
- **网络结构**：太小可能欠拟合，太大可能过拟合。128-256 hidden 通常够
- **探索不足**：策略熵是否过早到 0？ ϵ 是否衰减太快？
- **批次太小**：尝试 batch_size=128 或 256
- **梯度爆炸**：加入梯度裁剪 clip_grad_norm_(5.0)

14.2 Q 值爆炸

DQN 类算法常见症状：训练初期 Q 值合理，但某时刻突然飙升到 $1e10$ 。原因：max 操作放大了过估误差，与 bootstrapping 形成正反馈。解决方案：

- 用 Double DQN（最有效）
- 减小学习率 ($1e-3 \rightarrow 3e-4$)
- 加大 batch_size ($64 \rightarrow 256$)
- 梯度裁剪 clip_grad_norm_(10.0)
- Huber loss 替代 MSE（对异常值更鲁棒）

14.3 策略熵过早坍缩

A2C/PPO 训练中常见：策略很快变成确定性，停止探索，性能卡在局部最优。诊断：监控策略熵曲线，若 50 回合内熵降到接近 0 即为此问题。解决方案：

- 增大熵系数 ($0.01 \rightarrow 0.05$)
- 降低学习率（防止策略变化过快）
- 增加网络容量 ($128 \rightarrow 256$)
- 检查 GAE λ 是否过大 (>0.97 方差大)

14.4 Gymnasium 版本兼容

Gymnasium 0.26 引入了 API 变更，许多旧代码无法直接运行。关键改动：

- reset() 返回 (obs, info)，而非 obs
- step() 返回 (obs, reward, terminated, truncated, info)，5 元组而非 4 元组
- terminated: 自然终止（任务完成或失败）
- truncated: 因时间限制等外部条件终止
- Bootstrap 时只用 $(1 - \text{terminated})$ 作 mask，不用 truncated

```
# Gymnasium 0.26+ 正确用法
obs, info = env.reset(seed=42)
for _ in range(max_steps):
    action = agent.act(obs)
    obs, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    # TD 更新时用 terminated 作 mask (truncated 不影响 bootstrap)
    td_target = reward + gamma * Q(obs_next) * (1 - terminated)
    if done:
        break
```

14.5 GPU 显存爆炸

训练大网络时常见 OOM (Out of Memory)。排查与解决：

- 检查 batch_size, 常见 64-256
- 检查回放缓冲区是否在 GPU 上 (应在 CPU, 采样时再传 GPU)
- 检查是否累积了计算图 (torch.no_grad() / detach() 是否正确使用)
- 使用混合精度训练 (torch.cuda.amp)
- 分布式训练用 gradient accumulation 模拟大 batch

14.6 探索-利用失衡

若训练曲线在低位震荡或卡住, 可能是探索问题。诊断：

- **探索不足**：策略熵接近 0；不同 episode 行为几乎相同 → 加大探索
- **探索过度**：策略熵一直很高；eval 性能远高于 train → 减小探索

调节手段：

- ϵ -greedy: 调衰减速度, 最终值 0.05
- 熵正则: PPO/SAC 的 entropy_coef
- NoisyNet: 替代 ϵ -greedy
- Intrinsic motivation: ICM/RND 给探索额外奖励
- Curiosity-driven: 用预测误差作为探索奖励

14.7 评估与对比的常见误区

报告 RL 结果时容易犯的错误：

- **只看一条曲线**：必须多 seed (5+) 取平均, 画 shaded area (std)
- **报告 train 而非 eval**：eval 性能 (关闭探索) 才是策略真实性能
- **不公平对比**：不同算法的环境数、总步数应一致
- **选最好 seed**：cherry-picking 是学术不端；应报告所有 seed 的均值
- **不看样本效率**：算法 A 在 1M 步达到 500 分, 算法 B 在 100K 步达到 450 分——B 在实际中更有用

14.8 调试 checklist

开始新算法时，按以下 checklist 逐项检查：

- ☐ 环境正确加载，obs/action 维度对
- ☐ obs 标准化（连续控制必备）
- ☐ reward 范围合理（必要时 scaling）
- ☐ 网络初始化（推荐 orthogonal_）
- ☐ 优化器 Adam，lr 合适
- ☐ 梯度裁剪已加
- ☐ entropy bonus 已加（on-policy 算法）
- ☐ target 网络 / 软更新（off-policy）
- ☐ buffer 容量合理
- ☐ 训练曲线、损失曲线、Q/熵曲线都在监控
- ☐ 多 seed 评估

[技巧] RL 调试经验积累极其重要。建议每实现一个算法，记录下“踩过的坑”——下次复现或迁移到新任务时就能避免重蹈覆辙。这也是本教程反复强调“运行配套代码”的原因：只有亲手跑过、亲手调过，才能真正理解算法。

至此，本教程的 14 章正文全部结束。从最基础的 MDP 到前沿的 RLHF，从最简单的 Q-Learning 到工业级 PPO/SAC，我们走过了强化学习的完整知识图谱。配套的 12 份代码可以立即运行——建议读者从第一章开始，一边读代码一边对照本章理论，遇到问题再回头查阅相关章节。强化学习的世界很大，本教程只是入门；但它提供的知识框架与代码原型，足以支撑读者继续探索更深的研究方向与更复杂的实际应用。